

Guide Complet : UDP, TCP et Multicast

Introduction

Ce document présente les concepts fondamentaux des protocoles réseau **TCP**, **UDP** et **Multicast** en Java. Il répond aux questions posées durant notre discussion et fournit des explications simples avec exemples de code.

1. Stream vs Datagram

Qu'est-ce qu'un Stream (TCP) ?

Stream (TCP) utilise le protocole **TCP**, qui crée une connexion stable entre deux machines.

Caractéristiques :

- Les données circulent dans un flux **continu** et **ordonné** (comme un tuyau)
- Chaque paquet arrive dans l'ordre et sans perte grâce à des mécanismes automatiques de contrôle
- Accusé de réception et réémission automatique si erreur
- **Un seul canal** par connexion : toutes les données passent par ce flux logique unique

Usage typique : transfert de fichiers, navigation web, chat, email

Analogie : Tu envoies une lettre dans une enveloppe, tu attends que le facteur la livre et t'informe de la livraison.

Qu'est-ce qu'un Datagram (UDP) ?

Datagram (UDP) utilise le protocole **UDP**, qui n'établit pas de connexion.

Caractéristiques :

- Les données circulent sous forme de petits **paquets indépendants**, appelés *datagrammes*
- L'ordre des paquets n'est pas garanti et certains peuvent être perdus
- Pas de vérification de réception
- **Pas de canal unique** : chaque paquet suit sa propre route sur le réseau
- Rapidité prioritaire sur la fiabilité

Usage typique : jeux vidéo, streaming, appels vidéo/audio

Analogie : Tu jettes plusieurs cartes postales dans la boîte ; certaines arriveront, d'autres non, et l'ordre d'arrivée n'a pas d'importance.

Résumé Comparatif

Critère	TCP (Stream)	UDP (Datagram)
Connexion	Oui (établie)	Non (sans connexion)
Fiabilité	Garantie	Non garantie
Ordre des paquets	Préservé	Non préservé
Vitesse	Plus lent	Plus rapide
Canal	Un seul canal fixe	Plusieurs chemins possibles
Taille paquet	Illimitée (découpée)	Max 65 507 octets

Table 1: Comparaison TCP vs UDP

2. Connexion : TCP vs UDP

TCP : Clients DOIVENT être connectés

En TCP, chaque client établit une **connexion** avec le serveur. La classe Socket est utilisée : dès que le Socket est ouvert, le client est connecté.

Code exemple - Vérification connexion TCP :

```
Socket socket = new Socket("localhost", 1234);
if (!socket.isClosed()) {
    System.out.println("TCP : Connecté !");
} else {
    System.out.println("TCP : Déconnecté !");
}
```

Obligation : Le client doit toujours avoir sa connexion active pour communiquer.

UDP : Clients NE sont PAS connectés

En UDP, il n'y a **jamais de vraie connexion**. Le client crée un DatagramSocket, puis envoie/reçoit des paquets sans établir de canal permanent.

Code exemple - Vérification socket UDP :

```
DatagramSocket ds = new DatagramSocket();
if (!ds.isClosed()) {
    System.out.println("UDP : Socket prêt à envoyer (jamais vraiment connecté) !");
} else {
    System.out.println("UDP : Socket fermé !");
}
```

Remarque : Un client UDP n'est jamais vraiment "connecté" au serveur. Le socket est juste "ouvert" ou "fermé", mais pas "connecté" dans le sens TCP.

Vérification côté Serveur

Serveur TCP :

```
// Afficher tous les utilisateurs connectés
public static void printConnectedUsers() {
    System.out.println("=== Utilisateurs TCP connectés ===");
    for (ClientHandler c : clients)
        System.out.println("- @" + c.getDisplayName());
}
```

Serveur UDP :

```
// Afficher tous les clients ayant communiqué
System.out.println("=== Clients UDP ayant communiqué ===");
for (String userId : clients.keySet())
    System.out.println("- " + userId + " @" + clients.get(userId));
```

3. Paquets dans TCP et UDP

Les deux utilisent des paquets

Important : Les deux protocoles (TCP et UDP) utilisent des paquets sur le réseau parce que l'Internet transporte tout en petits morceaux appelés "paquets".

Différence de gestion

TCP (Stream) :

- Envoie un flux de données, mais pour le transport, il "découpe" ce flux en paquets IP
- Assemble/recompose le flux à l'arrivée
- Pour le programme Java, tu vois un "canal" et pas des paquets séparés
- **Découpage et réassemblage = automatique**

UDP (Datagram) :

- Tu envoies et reçois des "paquets" directement
- Ils ne sont pas assemblés automatiquement
- **Gestion des paquets = manuelle** (par le programmeur)

Résumé :

- **TCP** : paquets existent en interne, mais tu manipules un flux continu
- **UDP** : tu manipules directement les paquets (datagrammes)

4. Taille Maximale des Datagrammes UDP

Limite technique

Chaque paquet en Datagram (UDP) a une taille maximale d'environ **65 Ko** (65 507 octets exactement).

La taille maximale d'un DatagramPacket est de **65 507 octets** (légèrement moins que 65 536 à cause des en-têtes IP et UDP).

Si le paquet est trop gros

Si tu envoies un paquet plus gros :

- Il sera rejeté ou fragmenté par le réseau
- La réception n'est jamais garantie
- Il vaut mieux envoyer des paquets de moins de 65 507 octets

Code exemple :

```
byte[] data = new byte[65507]; // Taille max pour un DatagramPacket
DatagramPacket packet = new DatagramPacket(data, data.length, address, port);
```

Conseil : Toujours penser à découper les gros fichiers en plusieurs paquets UDP !

5. Canaux : Un ou Plusieurs ?

TCP : Un seul canal par connexion

En TCP, chaque connexion crée un **canal unique** (virtuel) entre deux machines :

- Toutes les données envoyées dans cette connexion passent par ce canal logique
- Ce canal est maintenu et ordonné du début à la fin de la connexion
- Comme un tuyau réservé entre client et serveur

UDP : Pas de canal unique

En UDP, chaque communication est indépendante :

- Chaque datagramme peut passer par différents chemins (= "canaux") sur le réseau
- Il n'y a **pas de canal réservé** ou continu entre deux machines
- On peut dire qu'il y a autant de "mini-canaux" (éphémères, changeants) que de paquets envoyés

Résumé :

- **UDP** : pas de canal unique, chaque datagramme suit sa route, communication sans tunnel fixe
- **TCP** : un seul canal par connexion, tout passe par ce flux

6. Multicast : UDP uniquement

Qu'est-ce que le Multicast ?

Multicast est une technique qui permet d'envoyer un même paquet à **plusieurs machines** (groupes d'utilisateurs) en une seule fois.

Multicast = UDP seulement

- **UDP Multicast** permet l'envoi à plusieurs destinataires simultanément
- Très utilisé pour : vidéo en direct, radio internet, jeux multijoueurs
- Pas de connexion, pas de fiabilité garantie, chaque récepteur reçoit indépendamment

TCP ne propose pas de multicast : il fonctionne uniquement en connexion "un à un" (1 client vers 1 serveur).

Résumé :

- **Multicast = UDP uniquement**
- TCP : pas de multicast, seulement du point-to-point

7. DatagramSocket vs MulticastSocket

DatagramSocket

C'est la **classe de base** pour UDP en Java.

Caractéristiques :

- Permet d'envoyer et de recevoir des paquets UDP entre deux machines
- Communication "classique" un à un (client ↔ serveur)
- Unicast uniquement

Code exemple :

```
DatagramSocket socket = new DatagramSocket(port);  
// Envoie et reçoit des DatagramPacket
```

MulticastSocket

C'est une **sous-classe** de DatagramSocket.

Caractéristiques :

- Permet d'envoyer des paquets UDP à un **groupe d'adresses** (multicast)
- Utile pour envoyer à **plusieurs clients en même temps**
- Gère l'adhésion à un "groupe multicast" (plage IP spéciale : 224.0.0.0 à 239.255.255.255)

Code exemple :

```
MulticastSocket mSocket = new MulticastSocket(port);  
// Pour rejoindre un groupe multicast :  
mSocket.joinGroup(InetAddress.getByName("224.0.0.1"));
```

Comparaison

Classe	Pour...	Unicast (1-1)	Multicast (1-N)
DatagramSocket	UDP classique	Oui	Non
MulticastSocket	UDP vers groupe	Oui	Oui

Table 2: DatagramSocket vs MulticastSocket

À retenir :

- `DatagramSocket` → simple (point à point)
- `MulticastSocket` → multicast (diffusion à un groupe)

8. InputStream/OutputStream vs Reader/Writer

InputStream / OutputStream : pour les octets (bytes)

Utilisé pour lire/écrire des **données binaires** (raw bytes).

Caractéristiques :

- Parfait pour : images, fichiers, sons, vidéos
- Méthodes : read(), write(byte[])
- Manipulation au niveau octet

Code exemple :

```
InputStream in = socket.getInputStream();  
OutputStream out = socket.getOutputStream();
```

Reader / Writer : pour les caractères (text)

Utilisé pour lire/écrire du **texte** (chaînes de caractères, String).

Caractéristiques :

- Parfait pour : messages texte, fichiers texte
- Méthodes : readLine(), write(String)
- Manipulation au niveau caractère avec encodage

Code exemple :

```
BufferedReader reader = new BufferedReader(  
    new InputStreamReader(socket.getInputStream())  
);  
BufferedWriter writer = new BufferedWriter(  
    new OutputStreamWriter(socket.getOutputStream())  
);
```

Tableau Récapitulatif

Pour...	Utilise...	Exemple Java
Données binaires (octets)	InputStream/OutputStream	in.read(byte[]), out.write(byte[])
Texte (caractères)	Reader/Writer	reader.readLine(), writer.write(str)

Table 3: Choix des flux selon le type de données

À retenir :

- **bits/octets** (byte[]) → InputStream/OutputStream
- **texte/caractères** (String) → Reader/Writer

9. Exemples de Code Complets

Serveur TCP : Vérification connexions

```
public class ServeurTCP {
    static final List<ClientHandler> clients = new CopyOnWriteArrayList<>();

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(1234)) {
            System.out.println("Serveur TCP démarré sur port 1234");

            while (true) {
                Socket socket = serverSocket.accept();
                ClientHandler handler = new ClientHandler(socket);
                clients.add(handler);
                new Thread(handler).start();

                System.out.println("Client connecté : " + socket.getInetAddress());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    // Afficher tous les clients connectés
    public static void printConnectedUsers() {
        System.out.println("=== Clients TCP connectés ===");
        for (ClientHandler c : clients) {
            System.out.println("- " + c.getDisplayName());
        }
    }
}
```

Serveur UDP : Gestion clients

```
public class ServeurUDP {
    private DatagramSocket socket;
    private Map<String, InetSocketAddress> clients = new HashMap<>();
```

```

public void demarrer() throws Exception {
    socket = new DatagramSocket(9876);
    System.out.println("Serveur UDP démarré sur port 9876");

    while (true) {
        byte[] buffer = new byte[65507];
        DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
        socket.receive(packet);

        String message = new String(packet.getData(), 0, packet.getLength());
        InetSocketAddress clientAddr = new InetSocketAddress(
            packet.getAddress(),
            packet.getPort()
        );

        // Enregistrer le client
        clients.put(message, clientAddr);

        System.out.println("Message reçu : " + message);
    }
}

// Afficher tous les clients ayant communiqué
public void printClients() {
    System.out.println("=== Clients UDP ===");
    for (String userId : clients.keySet()) {
        System.out.println("- " + userId);
    }
}
}

```

Client Multicast : Rejoindre un groupe

```

public class ClientMulticast {
    public static void main(String[] args) throws Exception {
        // Créer le socket multicast
        MulticastSocket socket = new MulticastSocket(8888);
    }
}

```



```

// Rejoindre le groupe multicast
InetAddress groupe = InetAddress.getByName("224.0.0.1");
socket.joinGroup(groupe);

System.out.println("Rejoint le groupe multicast 224.0.0.1");

// Recevoir des messages
byte[] buffer = new byte[1024];
while (true) {
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
    socket.receive(packet);

    String message = new String(packet.getData(), 0, packet.getLength());
    System.out.println("Message reçu : " + message);
}
}
}

```

10. Résumé Général

Points Clés à Retenir

- **TCP (Stream)** : connexion établie, fiable, ordonnée, un seul canal
- **UDP (Datagram)** : sans connexion, rapide, non fiable, pas de canal fixe
- **Multicast** : uniquement UDP, diffusion à un groupe
- **Taille max UDP** : 65 507 octets par paquet
- **DatagramSocket** : UDP classique (1-1)
- **MulticastSocket** : UDP multicast (1-N)
- **InputStream/OutputStream** : données binaires (bytes)
- **Reader/Writer** : données texte (caractères)

Quand Utiliser Quoi ?

Cas d'usage	Protocole recommandé
Chat texte fiable	TCP
Transfert de fichiers	TCP
Jeux vidéo en temps réel	UDP
Streaming vidéo/audio	UDP
Diffusion à plusieurs clients	UDP Multicast
Navigation web	TCP
Appels vidéo	UDP
Email	TCP

Table 4: Choix du protocole selon le cas d'usage

Conclusion

Ce guide a présenté les différences fondamentales entre TCP, UDP et Multicast. TCP offre la fiabilité avec une connexion établie et un canal unique, tandis qu'UDP privilégie la rapidité sans garantie de livraison. Le multicast, exclusif à UDP, permet la diffusion efficace à plusieurs destinataires simultanément.

Le choix entre ces protocoles dépend des besoins de l'application : fiabilité et ordre pour TCP, performance et temps réel pour UDP, et diffusion de groupe pour Multicast.